

tiny-solver-rs 功能對齊計劃

以 ceres-solver 為基準之完整功能覆蓋路線圖

版本	1.0	
日期	2026 年 8 月	
對象	tiny-solver-rs (Rust)	對照 ceres-solver (C++)

目錄

1	前言與目標	3
1.1	專案背景	3
1.2	「100% 功能對等」的定義與範疇	3
2	tiny-solver-rs 現況分析	3
2.1	模組架構總覽	3
2.2	已具備的能力	4
2.3	主要限制與缺口（總覽）	5
3	ceres-solver 功能盤點	5
3.1	CostFunction 與微分方式	5
3.2	損失函數（Loss Function）	6
3.3	流形（Manifold）	6
3.4	求解器與最佳化策略	6
3.5	線性代數與求解器族群	7
3.6	共變異數估計、梯度檢查與其他工具	7
3.7	效能與架構特性	7
3.8	ceres 內建的 tiny_solver（嵌入式求解器）	8
4	功能落差分析總表	8
5	分階段開發計劃	9
5.1	P0：求解器診斷與收斂準則對齊（基礎建設）	9
5.2	P1：損失函數補完	10
5.3	P2：流形（Manifold）補完	10
5.4	P3：微分方式擴充	10
5.5	P4：線性求解器與 Bundle Adjustment 專用結構	11
5.6	P5：Dogleg 信賴域策略	11
5.7	P6：Line Search Minimizer	11
5.8	P7：共變異數估計（Covariance）	12
5.9	P8：GradientChecker 與數值驗證工具	12
5.10	P9：GradientProblem／無結構純梯度最佳化	12
5.11	P10：影像插值工具（可選、依需求排序）	12
5.12	P11：效能與平行化強化	12
5.13	P12：Python 綁定功能對齊	13
5.14	P13：標準測試集與跨語言驗收	13
6	風險評估與因應策略	13
7	驗收標準（Definition of Done 總表）	14
8	結論	14

1 前言與目標

1.1 專案背景

tiny-solver-rs 是一套以 Rust 撰寫的非線性最小平方 (Nonlinear Least Squares) 求解器，架構上仿照 Google 的 ceres-solver，並直接借用其 `tiny_solver.h` 之命名精神：提供一個以 factor graph (殘差區塊 + 參數區塊) 為核心、支援自動微分、流形 (Manifold)、穩健損失函數 (Robust Loss) 與稀疏線性代數的最小平方求解框架，目前已可處理 Pose Graph Optimization (SE2 / SE3)、Bundle Adjustment 等常見應用 (見 `examples/m3500_benchmark.rs`、`examples/sphere2500.rs`、`examples/parking-garage.rs`)。

相較之下，ceres-solver 是歷經十餘年開發、在工業界 (Google Maps、SLAM、SfM、機器人) 被大量驗證的成熟函式庫，功能涵蓋範圍遠超過 tiny-solver-rs 目前的實作。本文件的目的，是在詳細閱讀兩者原始碼後，盤點兩者之功能差異，並提出一份可執行、按優先順序推進的開發計劃，讓 tiny-solver-rs 逐步達成與 ceres-solver 對等 (甚至在 Rust 生態下更安全、更易用) 的功能覆蓋率。

1.2 「100% 功能對等」的定義與範疇

ceres-solver 是一個涵蓋數十萬行程式碼、支援多種第三方稀疏代數後端 (SuiteSparse、Accelerate、Eigen)、多執行緒排程、GPU 相容數值精度切換等特性的工業級函式庫。逐行「複製」其實作既不可行，也不是 Rust 生態下最合適的手段。因此本計劃將「100% 功能對等」界定為「使用者可觀察到的能力 (capability) 對等」，而非「內部實作方式對等」，具體包含：

- 數學功能對等：所有 ceres-solver 提供的損失函數、流形、微分方式、求解策略、線性求解器族群，tiny-solver-rs 都能提供功能相同或效果等價的實作。
- API 能力對等：Problem/Solver 的操作介面 (新增/移除殘差區塊、鎖定變數、邊界限制、共變異數估計、回呼機制、診斷報告) 具備對應能力。
- 工程可用性對等：效能特性 (多執行緒、稀疏矩陣重用、Schur complement 化簡) 在同等問題規模下達到可接受的效能量級 (同一數量級)，不要求逐位元組相同。
- 驗證對等：以 ceres-solver 內建的標準測試問題 (NIST、Powell、More-Garbow-Hillstom、curve fitting、BAL 資料集) 作為雙方比對的驗收基準。

不在本計劃範圍內 (明確排除)：

- 第三方稀疏代數後端的原生繫結 (SuiteSparse/CHOLMOD、Apple Accelerate 的直接 FFI)，改以 Rust 原生的 faer 稀疏代數庫達成等價功能。
- C API (`c_api.h`) 與非 Rust 語言的直接綁定 (Python 綁定仍在範圍內，但以現有 tiny_solver PyO3 套件為主)。
- GPU 加速與混合精度矩陣分解的底層硬體特化 (僅規劃 CPU 對等的近似方案)。

2 tiny-solver-rs 現況分析

2.1 模組架構總覽

現有原始碼 (`src/`) 之模組劃分如下：

模組	職責
lib.rs	對外匯出各模組 (factors/linear/loss_functions/manifold/optimizer/parameter_block/problem/residual_block)
problem.rs	Problem 結構：管理殘差區塊、固定變數、邊界、流形；建構稀疏 Jacobian 的符號結構 (symbolic sparsity pattern) 並以 rayon 平行計算殘差與 Jacobian
residual_block.rs	單一殘差區塊：呼叫 <code>factors::FactorImpl</code> 計算殘差，並用 <code>num_dual::DualDVec64</code> 進行前向自動微分取得 Jacobian，套用穩健損失的 Corrector
parameter_block.rs	單一參數區塊：管理 ambient/tangent 維度、plus/minus、固定索引、邊界裁切、可選 Manifold
factors.rs	Factor<T: RealField> trait；內建 BetweenFactorSE2、BetweenFactorSE3、PriorFactor 三種因子
loss_functions.rs	Loss trait；內建 HuberLoss、CauchyLoss、ArctanLoss
corrector.rs	依據 ceres internal/ceres/corrector.cc 之演算法，將穩健損失的一階/二階資訊套用回殘差與 Jacobian
manifold/	Manifold trait (AutoDiffManifold<f64> + AutoDiffManifold<Dual>)；內建 SO3、SE3 流形
optimizer/	Optimizer trait；內建 GaussNewtonOptimizer、LevenbergMarquardtOptimizer；統一的 OptimizerOptions、apply_dx、compute_error
linear/	SparseLinearSolver trait；內建 SparseCholeskySolver、SparseQRSolver (皆基於 faer 稀疏矩陣運算)
helper.rs	g2o 檔案格式讀取，快速建立 Pose Graph 測試問題

2.2 已具備的能力

- 以 `num_dual::DualDVec64` 達成前向自動微分 (等價於 ceres 的 `Jet<T,N>`)，Jacobian 由 factor 的殘差函式自動推導，無需手寫解析導數。
- 支援固定變數 (`fix_variable`) 與逐元素邊界限制 (`set_variable_bounds`)，對應 ceres 的 `SetParameterBlockConstant` 與 `SetParameterUpperBound/LowerBound`。
- 支援自訂 Manifold (`set_variable_manifold`)，已提供 SO3/SE3 兩種流形。
- 具備三種穩健損失函數 (Huber/Cauchy/Arctan)，並正確實作 ceres 的 Corrector 演算法 (一階修正 Jacobian、殘差縮放)。
- 具備 Gauss-Newton 與 Levenberg-Marquardt 兩種最佳化策略，LM 版本包含 jacobian scaling、trust region 半徑調整 (rho 判斷) 等 ceres 對等邏輯。
- 具備稀疏 Cholesky/QR 兩種線性求解器 (faer 提供)，並以符號結構快取 (SymbolicStructure) 避免重複計算稀疏樣式。
- 以 rayon 平行化殘差與 Jacobian 計算 (區塊層級平行)。

2.3 主要限制與缺口（總覽）

以下缺口將在第 4 章展開為完整對照表，此處先列出影響最大的幾項：

1. 只有 LM/GN 兩種 trust-region 策略，無 Dogleg；完全沒有 Line Search Minimizer（Steepest Descent/CG/LBFGS/BFGS）。
2. 線性求解器只有稠密般的稀疏 Cholesky/QR，沒有 Bundle Adjustment 關鍵的 Schur Complement 化簡（DENSE_SCHUR/SPARSE_SCHUR/ITERATIVE_SCHUR），大規模 BA 問題效能會遠遜於 ceres。
3. 損失函數只有 3 種，缺少 SoftLOne、Tolerant、Tukey、Composed、Scaled、Trivial。
4. 流形只有 SO3/SE3，缺少通用 EuclideanManifold、QuaternionManifold、SphereManifold、ProductManifold、SubsetManifold、LineManifold。
5. 沒有共變異數估計（Covariance）、沒有 GradientChecker、沒有 Solver::Summary/FullReport 診斷輸出、沒有 IterationCallback/EvaluationCallback。
6. 只有前向自動微分一種微分方式，缺少數值微分（Forward/Central/Ridders）與「使用者自行提供解析 Jacobian」的正式介面，對無法自動微分的殘差函式（例如查表、外部程式呼叫）不友善。
7. 沒有 GradientProblem/GradientProblemSolver 等純梯度（無參數區塊結構）最佳化介面。
8. 沒有影像插值工具（Grid1D/Grid2D/CubicInterpolator/BiCubicInterpolator），無法直接支援以影像像素值為殘差的視覺任務（如光度誤差 BA）。

3 ceres-solver 功能盤點

本章彙整詳細閱讀 ceres-solver/include/ceres/*.h 與 ceres-solver/internal/ceres/* 後的功能盤點，作為第 4 章落差分析的基準。

3.1 CostFunction 與微分方式

- CostFunction 基底類別：使用者實作 Evaluate(parameters, residuals, jacobians)，Jacobian 可個別設為 nullptr 以跳過不需要的導數計算。
- AutoDiffCostFunction：以 Jet<T,N> 樣板達成前向自動微分，殘差函式以樣板 operator() 撰寫一次即可同時取得數值與導數。
- NumericDiffCostFunction：支援 Forward/Central/Ridders 三種數值微分法，可包裝既有 CostFunction，並提供相對/絕對步長選項（NumericDiffOptions）。
- DynamicAutoDiffCostFunction/DynamicNumericDiffCostFunction：參數區塊數量與大小可於執行期決定（Bundle Adjustment 中每個殘差牽涉的相機/點數量不固定時）。
- SizedCostFunction：提供固定殘差/參數維度但需使用者手寫解析 Jacobian 的基底類別。
- ConditionedCostFunction：對既有殘差逐元素套用 1x1 conditioner function（鏈式法則轉換 Jacobian）。
- NormalPrior： $||A(x-b)||^2$ 形式的先驗項，用於貝氏先驗約束。
- CostFunctionToFunctor/DynamicCostFunctionToFunctor：將既有 CostFunction 包裝為可在 AutoDiffCostFunction 中組合使用的 functor。

3.2 損失函數 (Loss Function)

損失函數	公式 / 用途
TrivialLoss	$\rho(s) = s$ ，等同於不使用穩健損失
HuberLoss(a)	$s \leq a^2$ 線性； $s > a^2$ 平方根成長，經典穩健損失
SoftLOneLoss(a)	$\rho(s) = 2(\sqrt{1+s} - 1)$ ，Huber 的平滑近似版本
CauchyLoss(a)	$\rho(s) = \log(1+s)$ ，對離群值更激進的抑制
ArctanLoss(a)	$\rho(s) = a \cdot \arctan(s/a)$ ，隨 s 增大有上界
TolerantLoss(a,b)	容忍區間 $[a-b, a+b]$ 內近乎線性，超出後平滑過渡
TukeyLoss(a)	超過門檻後代價完全飽和（趨近常數），抑制極端離群值
ComposedLoss(f,g)	兩個損失函數的組合 $\rho(s) = f(g(s))$
ScaledLoss(rho,a)	對既有損失函數的輸出做整體縮放

3.3 流形 (Manifold)

流形	Ambient	Tangent	用途
EuclideanManifold<N>	N	N	一般歐氏空間，等同無流形的預設行為
QuaternionManifold	4	3	Hamilton 四元數，單位範數約束 (w,x,y,z)
EigenQuaternionManifold	4	3	Eigen 排列的四元數 (x,y,z,w)
SphereManifold<D>	D	D-1	單位球面，常用於齊次座標點的規範化
LineManifold<D>	2D	2(D-1)	以 (原點,方向) 參數化的直線流形
ProductManifold	Σ Ambient	Σ Tangent	多個流形的笛卡兒積 (如 $SO(3) \times \mathbb{R}^3$ 之剛體變換)
SubsetManifold	N	N-k	固定其中 k 個維度、其餘可變

3.4 求解器與最佳化策略

- MinimizerType : TRUST_REGION (預設) / LINE_SEARCH。
- TrustRegionStrategyType : LEVENBERG_MARQUARDT / DOGLEG (TRADITIONAL_DOGLEG、SUBSPACE_DOGLEG)。
- LineSearchDirectionType : STEEPEST_DESCENT、NONLINEAR_CONJUGATE_GRADIENT (Fletcher-Reeves / Polak-Ribiere / Hestenes-Stiefel)、LBFGS (預設)、BFGS。
- LineSearchType : ARMIJO (backtracking) / WOLFE (預設，強 Wolfe 條件，More-Thuente 演算法)，插值方式 BISECTION / QUADRATIC / CUBIC。
- 收斂條件 : function_tolerance、gradient_tolerance、parameter_tolerance，皆為相對/絕對混合條件。
- Trust Region 控制 : 初始 / 最大 / 最小半徑、min_relative_decrease、max_num_consecutive_invalid_steps、use_nonmonotonic_steps。
- Inner Iterations : 針對可分離最小平方問題 (Ruhe & Wedin Algorithm II) 在每次外層迭代中對獨立參數子集做內層最佳化。

- 回呼機制：IterationCallback（可中止/提前成功結束）、EvaluationCallback（評估前共用暫存計算）。
- Solver::Summary／FullReport：完整的每次迭代診斷（cost、gradient norm、trust region 半徑、耗時）與終止原因報告。

3.5 線性代數與求解器族群

LinearSolverType	說明
DENSE_QR	稠密 QR 分解，適合小型問題
DENSE_NORMAL_CHOLESKY	對 $J^T J$ 做稠密 Cholesky 分解
SPARSE_NORMAL_CHOLESKY	對 $J^T J$ 做稀疏 Cholesky（AMD/NESDIS 排序）
DENSE_SCHUR	對 Bundle Adjustment 結構做稠密 Schur complement 化簡
SPARSE_SCHUR	對 BA 結構做稀疏 Schur complement 化簡
ITERATIVE_SCHUR	在 Schur complement 上以共軛梯度法隱式求解（免顯式組裝）
CGNR	對一般法方程式做共軛梯度法（Conjugate Gradients on Normal Residuals）

前置條件器（Preconditioner）包含 IDENTITY、JACOBI、SCHUR_JACOBI、SCHUR_POWER_SERIES_EXPANSION、CLUSTER_JACOBI、CLUSTER_TRIDIAGONAL、SUBSET。稀疏代數後端可選 SUITE_SPARSE、ACCELERATE_SPARSE、EIGEN_SPARSE。

3.6 共變異數估計、梯度檢查與其他工具

- Covariance：以

$$(J^T J)^{\{-1\}}$$

（或秩虧時的擬似逆）估計參數共變異數，支援僅計算使用者關心的區塊，並可處理固定參數/零維流形造成的秩虧。

- GradientChecker：比較解析/自動微分 Jacobian 與數值微分 Jacobian 的差異，並正確處理 Manifold 的投影 Jacobian。
- GradientProblem／GradientProblemSolver：無參數區塊結構、單純 $f(x) \rightarrow \text{scalar}$ 的最佳化介面，共用 Line Search Minimizer。
- rotation.h：AngleAxis $\leftrightarrow$ 四元數 $\leftrightarrow$ 旋轉矩陣的雙向轉換工具（含 Jet 相容版本）。
- cubic_interpolation.h：Grid1D/Grid2D + CubicInterpolator/ BiCubicInterpolator，用於以影像像素值為殘差的視覺任務。
- ordered_groups.h（ParameterBlockOrdering）：手動指定 Schur 消去順序（群組 0 通常為 3D 點、群組 1 為相機）。

3.7 效能與架構特性

- 多執行緒排程（num_threads）平行化殘差/Jacobian 求值與稀疏分解。
- Jacobian scaling（欄正規化）、dynamic_sparsity（每次迭代重新分解稀疏樣式）。
- 混合精度求解（use_mixed_precision_solves）：以 float 分解、double 精化。
- Block Sparse Matrix 內部表示，針對 BA 之相機/點二分結構最佳化。

3.8 ceres 內建的 tiny_solver (嵌入式求解器)

ceres 本身附帶一個極簡的 `tiny_solver.h`：僅使用 Eigen、零堆積配置、僅支援 Levenberg-Marquardt，透過樣板特徵 (TinySolverCostFunctionTraits) 在編譯期決定殘差/參數維度，適合嵌入式或即時應用。tiny-solver-rs 目前的定位介於此「極簡嵌入式求解器」與完整版 ceres 之間；本計劃將把它推向功能完整版，但仍會保留一個輕量化的「no_std 友善」精簡模式作為子選項（見 9.9 節）。

4 功能落差分析總表

下表以 ceres-solver 功能為列，標示 tiny-solver-rs 現況與後續規劃階段（見第 5 章）。

功能類別	具體項目	現況	規劃階段
微分方式	前向自動微分 (Dual number)	已具備	-
微分方式	數值微分 (Forward/Central/Ridders)	尚未支援	P3
微分方式	使用者手寫解析 Jacobian 介面	尚未支援	P3
微分方式	動態參數數量的殘差區塊	尚未支援	P3
損失函數	Huber / Cauchy / Arctan	已具備	-
損失函數	Trivial / SoftLOne / Tolerant / Tukey	尚未支援	P1
損失函數	Composed / Scaled 組合器	尚未支援	P1
流形	SO3 / SE3	已具備	-
流形	EuclideanManifold (顯式)	尚未支援	P2
流形	QuaternionManifold / EigenQuaternionManifold	尚未支援	P2
流形	SphereManifold	尚未支援	P2
流形	ProductManifold	尚未支援	P2
流形	SubsetManifold	尚未支援	P2
流形	LineManifold	尚未支援	P2
最佳化策略	Gauss-Newton / Levenberg-Marquardt	已具備	-
最佳化策略	Dogleg (Traditional / Subspace)	尚未支援	P5
最佳化策略	Line Search Minimizer (Steepest/CG/LBFGS/BFGS)	尚未支援	P6
最佳化策略	Inner Iterations (可分離子問題)	尚未支援	P6
終止條件	絕對/相對誤差變化	已具備	-
終止條件	gradient_tolerance / parameter_tolerance	部分具備	P0
線性求解器	Sparse Cholesky / Sparse QR	已具備	-
線性求解器	Dense QR / Dense Normal Cholesky	尚未支援	P4
線性求解器	Dense/Sparse Schur Complement (BA 專用)	尚未支援	P4
線性求解器	Iterative Schur + 前置條件器	尚未支援	P4
線性求解器	CGNR	尚未支援	P4

功能類別	具體項目	現況	規劃階段
線性求解器	Fill-reducing ordering (AMD/NESDIS)	部分具備	P4
Problem API	固定變數 / 邊界限制	已具備	-
Problem API	自訂 Manifold 綁定	已具備	-
Problem API	殘差區塊快速移除 / 查詢 API	部分具備	P0
Problem API	ParameterBlockOrdering (Schur 消去順序)	尚未支援	P4
診斷與回呼	Solver::Summary / FullReport	尚未支援	P0
診斷與回呼	IterationCallback / EvaluationCallback	尚未支援	P0
進階分析	Covariance 估計	尚未支援	P7
進階分析	GradientChecker	尚未支援	P8
進階功能	GradientProblem / GradientProblemSolver	尚未支援	P9
進階功能	Cubic/BiCubic 影像插值	尚未支援	P10
效能	多執行緒 (殘差/Jacobian 平行)	已具備	-
效能	Dynamic sparsity 重新分解	尚未支援	P11
效能	混合精度求解	尚未支援	P11 (低優先)
綁定與工具	Python 綁定 (PyO3)	部分具備	P12
測試與驗證	NIST / Powell / MGH 標準測試集	尚未支援	P13

5 分階段開發計劃

本計劃刻意不採用「以週/月為單位」的時程表，而以優先級階段 (P0 至 P13) 排序，理由是各項工作的實際工時高度取決於實作者熟悉度與測試迭代次數；以下每個階段皆包含「目標」「工作項目」「完成定義 (Definition of Done)」，可作為 issue/milestone 拆分之依據，按序推進即可。

5.1 P0：求解器診斷與收斂準則對齊 (基礎建設)

目標：在不擴充新演算法的前提下，先讓既有 LM/GN 求解器的「可觀測性」與收斂行為與 ceres 對齊，這是後續所有階段的共同基礎。

工作項目：

- 於 `OptimizerOptions` 新增 `gradient_tolerance`、`parameter_tolerance`，並在 `LevenbergMarquardtOptimizer`/`GaussNewtonOptimizer` 中依 ceres 的定義計算 $\max_i |\text{gradient}_i|$ 與 $\|\text{dx}\|_2 \leq \text{tol} \times (\|x\|_2 + \text{tol})$ 兩項判斷。
- 新增 `IterationCallback` trait 與 `CallbackReturnType` (`Continue`/`TerminateSuccessfully`/`Abort`)，於每次迭代結束後呼叫。
- 新增 `Solver::Summary` 對等結構：記錄每次迭代的 `cost`、`gradient norm`、`trust region 半徑`、`耗時`，並提供 `full_report()` 產生可讀字串。

- 補齊 Problem 的查詢 API: num_parameter_blocks、num_residual_blocks、parameter_block_size、is_parameter_block_constant 等，對齊 problem.h 的唯讀查詢介面。
- 為 remove_residual_block 增加 O(1) 快速移除模式的選項說明與效能測試。

完成定義：既有範例 (m3500_benchmark、sphere2500、parking-garage) 在加入新收斂條件後結果不回歸；新增單元測試驗證 full_report() 輸出格式與終止原因。

5.2 P1：損失函數補完

工作項目：

- 新增 TrivialLoss、SoftL0Loss、TolerantLoss、TukeyLoss 四種損失函數，公式與二階導數依第 3.2 節表列實作。
- 新增 ComposedLoss<F, G> 與 ScaledLoss<L> 兩個組合器，使 Loss trait 可組合疊加。
- 依 ceres loss_function_test.cc 的數值案例撰寫對應單元測試 (tests/test_loss.rs 擴充)。

完成定義：8 種損失函數 + 2 種組合器全數通過與 ceres 參考數值一致 (誤差 < 1e-9) 的單元測試。

5.3 P2：流形 (Manifold) 補完

工作項目：

- 新增泛型 EuclideanManifold<N> (作為「無流形」情境的顯式版本，方便統一介面)。
- 新增 QuaternionManifold (w,x,y,z) 與 EigenQuaternionManifold (x,y,z,w)，以 num_dual 自動微分推導 Plus/Minus。
- 新增 SphereManifold<D> (單位球面投影)。
- 新增 SubsetManifold (固定部分維度的通用版本，取代目前 fixed_variables 只作用於「無流形參數」的限制，讓固定維度與流形可以並存)。
- 新增 ProductManifold，可將既有流形 (如 SO3 × EuclideanManifold<3>) 組合為單一參數區塊的流形，取代目前 SE3 專用寫死實作，讓使用者可自由組合。
- 新增 LineManifold<D> (可選，視實際需求優先度可下修)。

完成定義：g2o Pose Graph 範例可改用 ProductManifold(SO3 × 平移) 重現與現有 SE3Manifold 相同的最佳化結果；新增流形皆有對應 test_manifold.rs 測試，含 Plus/Minus 互逆性與數值 Jacobian 交叉驗證。

5.4 P3：微分方式擴充

工作項目：

- 新增 NumericDiffFactor 包裝器：對只實作 f64 版本殘差函式 (未實作 FactorImpl 的自動微分版本) 的使用者，提供 Forward/Central/Ridders 三種數值微分模式計算 Jacobian，並提供步長參數 (對齊 NumericDiffOptions)。
- 新增「解析 Jacobian」介面：允許使用者針對效能關鍵的殘差區塊直接實作 residual_and_jacobian_analytic，略過自動/數值微分。
- 新增 DynamicFactor：支援執行期決定參數區塊數量的殘差 (現有介面 variable_key_size_list: &[&str] 已支援動態長度，需補上對應文件與範例，並確認與 Jacobian 稀疏樣式建構的相容性)。

完成定義：新增 `tests/test_factors.rs` 案例比對三種微分方式與自動微分在同一殘差函式下的一致性；提供至少一個使用解析 Jacobian 的範例並量測其效能提升。

5.5 P4：線性求解器與 Bundle Adjustment 專用結構

此階段是達成大規模問題（如 BAL 資料集）效能對等的關鍵，優先度高但工程量最大，建議拆分為子階段推進：

- P4-a：稠密求解器：以 `faer` 稠密 QR/Cholesky 新增 `DenseQRSolver`、`DenseNormalCholeskySolver`，供小型稠密問題使用（可作為數值正確性的對照基準）。
- P4-b：Schur Complement 化簡：實作 `SchurComplementSolver`，依 `ParameterBlockOrdering` 將參數分為「可消去集合」（如 BA 中的 3D 點）與「保留集合」（相機參數），組裝縮減後的 Schur 系統，分別提供 `DenseSchur`（保留集合較小時）與 `SparseSchur`（保留集合較大時）兩種模式。
- P4-c：Iterative Schur + 前置條件器：以共軛梯度法在隱式 Schur 系統上求解，避免顯式組裝；先實作 `Jacobi`/`SchurJacobi` 前置條件器，`ClusterJacobi` 等視覺化叢集類前置條件器列為本階段延伸項目（優先度可下修）。
- P4-d：CGNR：對一般法方程式的共軛梯度求解器，作為稀疏 Cholesky 之外的備援選項（適合超大型、記憶體受限場景）。
- P4-e：Ordering：實作 `ParameterBlockOrdering`，讓使用者可手動指定 Schur 消去群組；內部排序（AMD 等價）已由 `faer` 稀疏分解自動處理，需驗證其 fill-reducing 效果與 `ceres` 的 AMD/NESDIS 相近。

完成定義：以 `ceres-solver/data/nist` 與 BAL 資料集 (`bundle_adjuster.cc` 對應資料) 建立跨語言效能與精度比較報告，Schur 版本在同規模 BA 問題上求解時間與 `ceres` 落在同一數量級（2-3 倍內可接受，作為 Rust 生態下的合理起點）。

5.6 P5：Dogleg 信賴域策略

工作項目：

- 新增 `TrustRegionStrategy` trait，將現有 LM 邏輯重構為其中一種實作 (`LevenbergMarquardtStrategy`)。
- 新增 `DoglegStrategy`，實作 Traditional Dogleg (Cauchy 點與 Gauss-Newton 步的分段線段插值)；Subspace Dogleg 視情況列為延伸項目。

完成定義：`OptimizerOptions` 可切換 LM / Dogleg，兩者在既有測試問題上收斂到相同最優解；新增針對病態 (ill-conditioned) 測試問題比較兩策略的穩健性。

5.7 P6：Line Search Minimizer

工作項目：

- 新增 `LineSearchMinimizer`，支援方向策略 `SteepestDescent`、`NonlinearConjugateGradient` (Fletcher-Reeves/Polak-Ribiere)、LBFGS。BFGS (稠密版本) 列為延伸項目。
- 新增 `LineSearch` trait，實作 Armijo (backtracking) 與 Wolfe (強 Wolfe 條件，可先以二次/三次插值取代 More-Thuente 的完整實作)。
- 新增 Inner Iterations (可分離子問題最佳化)，供 Pose Graph 一類具區塊獨立結構的問題使用。

完成定義：以 `ceres more_garbow_hillstrom.cc`/`powell.cc` 對應之標準測試函式驗證 LBFGS 與現有 LM 在無殘差 Jacobian 結構、僅有梯度資訊時亦可收斂。

5.8 P7：共變異數估計 (Covariance)

工作項目：

- 新增 Covariance API：於求解完成後，以最終 Jacobian 組裝 $J^T J$ ，透過 `faer` 稀疏 Cholesky 或稠密逆矩陣估計指定參數區塊組合的共變異數。
- 處理秩虧情況（固定參數、零維流形）之退化處理，對齊 `ceres` 的錯誤回報行為。

完成定義：以 `curve_fitting` 範例驗證估計出的參數共變異數與解析解（已知數據產生模型）一致；新增文件說明共變異數估計的假設（觀測值獨立同分布誤差）。

5.9 P8：GradientChecker 與數值驗證工具

工作項目：

- 新增 GradientChecker：對任一 Factor 實作，比較其自動微分 Jacobian 與數值微分 Jacobian（沿用 P3 的數值微分模組），輸出逐元素誤差報告。
- 整合進 `cargo test`：可作為使用者自訂 factor 的合約測試（contract test）範本釋出。

完成定義：內建三種既有 factor（`BetweenFactorSE2`/`BetweenFactorSE3`/`PriorFactor`）皆通過梯度檢查器交叉驗證，作為範例與回歸測試。

5.10 P9：GradientProblem／無結構純梯度最佳化

工作項目：

- 新增 GradientProblem 與對應 solver 入口，允許使用者直接提供 $f(x) \rightarrow (\text{cost}, \text{gradient})$ ，重用 P6 的 Line Search Minimizer 基礎設施，不需要透過殘差區塊/Jacobian 架構。

完成定義：以 `ceres rosenbrock` 一類的經典無約束最佳化測試函式驗證收斂正確性。

5.11 P10：影像插值工具（可選、依需求排序）

工作項目：

- 新增 `Grid1D`/`Grid2D` 抽象與 `CubicInterpolator`/`BiCubicInterpolator`，供以影像像素灰階值作為殘差的視覺任務（光度誤差、稠密對齊）使用。

完成定義：提供一個以合成影像做子像素對齊的範例，驗證插值精度與導數正確性。

5.12 P11：效能與平行化強化

工作項目：

- 支援 `dynamic_sparsity`：允許殘差結構於迭代間變動時重新建構符號結構，而非僅在第一次迭代建立（目前 `SymbolicStructure` 為一次性建構，需確認變動結構情境下的重建策略與效能取捨）。
- 評估混合精度求解（f32 分解 + f64 迭代精化）在 `faer` 上的可行性，列為本計劃中優先度最低的項目。
- 針對 P4 的 Schur Complement 化簡導入額外的區塊層級平行化。

完成定義：以 `sphere2500`/`BAL` 資料集做效能基準測試（benchmark），確認平行化後的加速比隨核心數合理擴展（非線性但正向）。

5.13 P12：Python 綁定功能對齊

工作項目：

- 隨 P1 – P9 新增的 Rust 端功能，同步擴充 `tiny_solver/PyO3` 綁定（`factors/`、`loss_functions/`、`tiny_solver.pyi` 型別定義）。

完成定義：Python 測試（`examples/python`）涵蓋所有新增的公開 API，`tiny_solver.pyi` 與實際綁定簽章一致（以型別檢查工具驗證）。

5.14 P13：標準測試集與跨語言驗收

工作項目：

- 移植 `ceres data/nist`、`examples/more_garbow_hillstrom.cc`、`examples/powell.cc`、`examples/curve_fitting.cc` 對應的標準測試問題至 `tests/`，並與 `ceres` 原生執行結果比對最終 `cost` 與參數解。
- 建立 BAL（Bundle Adjustment in the Large）資料集的端對端效能/精度報告，作為每次重大版本發布前的回歸基準。

完成定義：建立 CI 中的「跨語言一致性」測試套件，每次 release 前自動執行並產出報告。

6 風險評估與因應策略

風險	說明	因應策略
Rust trait object 動態派發開銷	<code>Box<dyn FactorImpl></code> ／ <code>Box<dyn Loss></code> 等動態派發在極大規模 BA 問題中可能造成 C++ 樣板靜態派發沒有的額外開銷	針對效能關鍵路徑（Schur Complement 組裝、殘差批次求值）評估改用 enum 派發或泛型單型化（monomorphization）版本作為進階選項
自動微分僅支援一階前向模式	<code>num_dual::DualDVec64</code> 僅提供一階導數，Dogleg／二階方法或需要 Hessian 的分析（如某些共變異數修正）需額外設計	優先以 Gauss-Newton 近似 Hessian ($J^T J$) 滿足現有與規劃中演算法的需求，避免真正引入二階自動微分的複雜度
稀疏代數後端限縮於 <code>faer</code>	不使用 <code>SuiteSparse</code> ／ <code>CHOLMOD</code> 意味著大型稀疏 Cholesky 的效能與排序演算法成熟度可能落後 <code>ceres</code> 預設後端	以 P4/P13 的跨語言效能報告持續追蹤落差； <code>faer</code> 為活躍維護的原生 Rust 稀疏代數庫，長期可受益於其自身效能優化
Schur Complement 實作複雜度高	P4 階段涉及大量索引簿記（消去順序、區塊稀疏結構），實作錯誤不易察覺	先以小規模合成 BA 問題（已知解析解）建立回歸測試，再逐步擴大規模至 BAL 真實資料集
功能擴充導致 API 破壞性變更	新增 <code>TrustRegionStrategy</code> ／ <code>LineSearchMinimizer</code> 等 trait 可能需要調整既有 <code>Optimizer</code> trait 簽章	以 semver 主版本號（1.0 → 2.0）規劃破壞性變更節點，並保留舊版 <code>GaussNewtonOptimizer</code> ／ <code>LevenbergMarquardtOptimizer</code> 作為相容包裝

風險	說明	因應策略
驗收基準取得困難	部分 ceres 測試資料 (NIST/BAL) 檔案龐大或授權需個別確認	優先使用 repo 內已附之 <code>ceres-solver/data/nist</code> 、 <code>ceres-solver/data/problem-16-22106-pre.txt</code> 等既有資料，避免額外下載

7 驗收標準 (Definition of Done 總表)

1. 正確性：每一階段新增功能皆有對應單元測試，且與 `ceres-solver` 之參考數值誤差小於 10^{-6} （一階導數）或 10^{-9} （損失函數數值）。
2. 回歸：既有三個範例 (`m3500_benchmark`、`sphere2500`、`parking-garage`) 在每次階段完成後仍可正確收斂，最終 `cost` 不劣化。
3. 效能：P4 (Schur Complement) 與 P11 (平行化) 完成後，於 BAL 中大型資料集上之求解時間與 `ceres-solver` 相差在同一數量級內。
4. 文件：每個新增的 public API (`trait`/`struct`/`enum`) 皆有對應 `rustdoc` 說明其與 `ceres-solver` 對應概念的映射關係，方便熟悉 `ceres` 的使用者遷移。
5. 綁定：Python 綁定 (P12) 與 Rust 端功能保持同步，型別定義檔正確。
6. 跨語言驗收 (P13)：建立可重複執行的跨語言比對測試套件並納入 CI。

8 結論

`tiny-solver-rs` 目前已具備 `ceres-solver` 核心「非線性最小平方 + 自動微分 + 穩健損失 + 流形 + 稀疏 LM 求解」的骨幹能力，這是最困難的架構決策部分已經到位；真正的落差集中在廣度（更多損失函數與流形變體）與大規模問題的專用結構（Schur Complement、Iterative Schur、多種前置條件器）以及工程可用性（診斷報告、回呼機制、共變異數估計、梯度檢查器）三個面向。

本計劃將工作拆分為 P0 至 P13 共 14 個優先級階段，建議依序推進，並在每個階段結束後以既有範例與新增測試作為回歸關卡。若資源有限，P0 (診斷基礎建設) 與 P4 (Schur Complement / Bundle Adjustment 專用結構) 應列為最高優先，前者是所有後續工作的可觀測性基礎，後者是決定 `tiny-solver-rs` 能否真正取代 `ceres-solver` 處理大規模 Bundle Adjustment 場景的關鍵。